Report: Casting Defect Detection

Project Stages

Stage 1 — Business s Data Understanding, Quality, Versioning, Methodology [20 Marks]

Task 1.1 — Business s Data Understanding [5 Marks]

- **1.1.1 – Business objective s justification [2]**

  Manual inspection of manufactured castings is slow, expensive, and prone to inconsistency. A computer-vision system can automatically identify defective parts, improving throughput and quality. However, deploying a model along is insufficient – the system must be operationally robust. Camera, lighting, and production lines evolve over time, so the solution must detect distribution shifts in incoming images, retrain safely when needed, and roll back if new model underperforms.

  Two challenges define this problem:

  - **Asymmetric error cost:** A missed defect (false negative) means a faulty part is shipped, which is far more costly than a false alarm. The system must therefore prioritize **recall on defects** over raw accuracy.
  - **Hidden distribution shifts:** Pixel-level checks are not enough. Monitoring must capture both interpretable image statistics (brightness, contrast, sharpness) and deeper feature-space representations (embeddings).

  This capstone is farmed as an **MLOps problem** – not just building a classifier, but designing the operational machinery that allows a factory to trust, serve, monitor, and maintain the model in production.

- ### 1.1.2 — Success metric choice (recall/F1 on defects) [2]

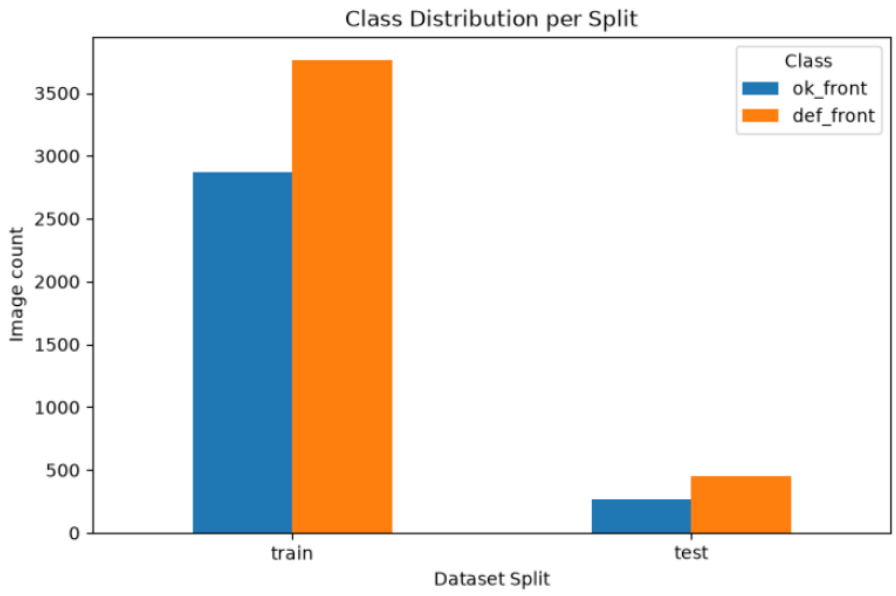
  The headline metric is recall **on the defect class**, supported by F1 and ROC-AUC. This choice reflects the business cost structure.

  - **Recall on defects:** Ensures defective parts are caught, minimizing costly false negatives.
  - **F1 and ROC-AUC:** Balance precision against recall and provide threshold-independent evaluation.
  - **Confusion matrix + failure analysis:** Reveal which defect types are being missed and guide improvements.
  - **Accuracy is explicitly rejected** as a selection metric, since it can mask missed defects and give a misleading sense of performance.

  By aligning metrics with the operational risks, the evaluation framework ensures the system delivers genuine business value rather than superficial accuracy gains.

- ### 1.1.3 — Dataset structure s class understanding [1]

train split: ok_front class count 2875, def_front class count 3758
test split: ok_front class count 262, def_front class count 453
Total Images: 7348
Positive Class: def_front, **Total Classes:** ['ok_front', 'def_front']
Total Classes Count: 2



Task 1.2 — Data Quality Validation [5 Marks]

- **1.2.1 — Corrupt-file s image-dimension checks [2]**
- **1.2.2 — Duplicate detection (hashing) — leakage guard [2]**
- **1.2.3 — Class distribution + pass/fail report [1]**

```
# TODO 1.2.1-1.2.3: qc = data_prep.validate_quality(root); print(qc['total_images'], qc['passed'])

qc = data_prep.validate_quality(root)
print("Total Images:", qc['total_images'])
print("Passed:", qc['passed'])
print("Class distribution:", qc['class_distribution'])
print("Corrupt files:", len(qc['corrupt_files']))
print("Bad dimensions:", len(qc['bad_dimensions']))
print("Duplicates:", len(qc['duplicates']))

print("\nWhy duplicate detection prevents train/test leakage?")
print(f"""
Duplicates across train/test splits cause leakage, inflating metrics unrealistically. Hashing ensures leakage-safety.
Even if duplicates inflate accuracy, recall on defects is the true business metric.

Duplicate images were detected via md5 hashing (~{len(qc['duplicates'])} duplicates).
These are flagged in the quality report but not deleted, since the requirement was detection only.
The rationale is to highlight potential train/test leakage, which can inflate metrics.
Later stages proceed with the dataset intact, but the report notes why duplicates matter.
""")
```

```
Total Images: 7348
Passed: False
Class distribution: {'train': {'ok_front': 2875, 'def_front': 3758}, 'test': {'ok_front': 262, 'def_front': 453}}
Corrupt files: 0
Bad dimensions: 0
Duplicates: 64
```

Total Images: 7348
Passed: False
Class distribution:
{'train': {'ok_front': 2875, 'def_front': 3758}, 'test': {'ok_front': 262, 'def_front': 453}}
Corrupt files: 0,
Bad dimensions: 0
Duplicates: 64

Why duplicate detection prevents train/test leakage?

- Duplicates across train/test splits cause leakage, inflating metrics unrealistically. Hashing ensures leakage-safety.
- Even if duplicates inflate accuracy, recall on defects is the true business metric.

Duplicate images were detected via md5 hashing (~64 duplicates). These are flagged in the quality report but not deleted, since the requirement was detection only. The rationale is to highlight potential train/test leakage, which can inflate metrics. Later stages proceed with the dataset intact, but the report notes why duplicates matter.

Task 1.3 — Dataset Versioning s Metadata [5 Marks]

- **1.3.1 — Reproducible, stratified train/val/test split [3]**
- **1.3.2 — Split snapshot + metadata.json (versioning) [2]**

Created reproducible stratified splits with a fixed seed to preserve class balance. Each split was snapshot into JSON files, and metadata.json records version, date, class definitions, and split sizes. This ensures auditability and leakage-safety.

{

"train": {
"ok_front": 2444,
"def_front": 3194
},
"val": {
"ok_front": 431,
"def_front": 564
},
"test": {
"ok_front": 262,
"def_front": 453
}

}



```
# TODO 1.3.1-1.3.2: meta = data_prep.build_splits(root, 'v1'); print(meta['split_info'])
meta = data_prep.build_splits(root, 'v1')
print(json.dumps(meta['split_info'], indent=2))

print("""
Created reproducible stratified splits with a fixed seed to preserve class balance.
Each split was snapshot into JSON files, and metadata.json records version, date, class definitions, and split sizes.
This ensures auditability and leakage-safety.
""")
```


```
{
  "train": {
    "ok_front": 2444,
    "def_front": 3194
  },
  "val": {
    "ok_front": 431,
    "def_front": 564
  },
  "test": {
    "ok_front": 262,
    "def_front": 453
  }
}
```

Task 1.4 — EDA s Methodology Design [5 Marks]

- **1.4.1 — Image-characteristic EDA [3]**

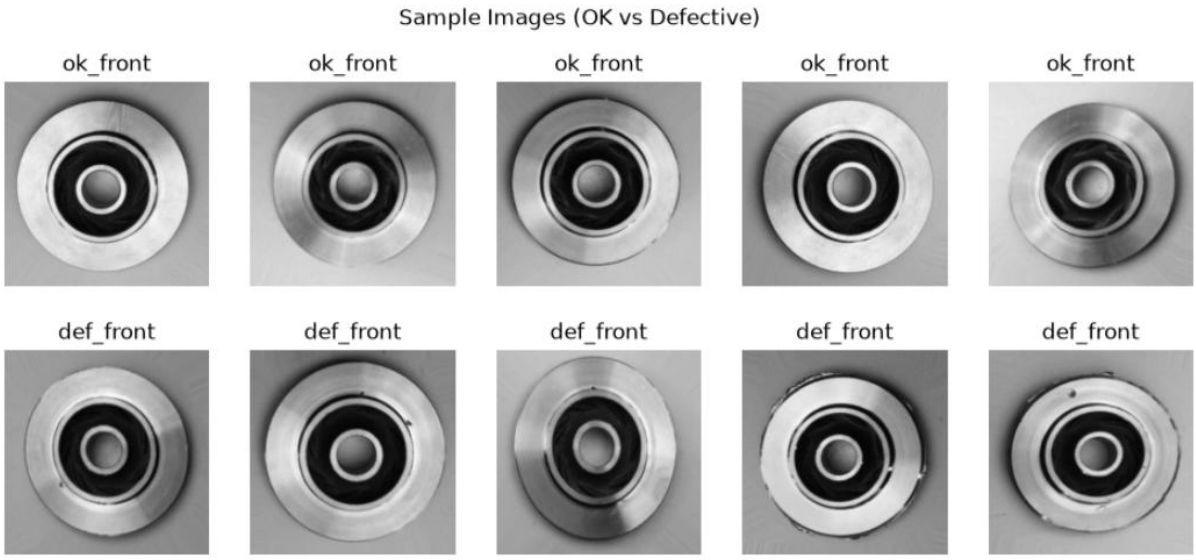
Interpretation:

- **OK vs Defective images differ in brightness, contrast, and sharpness.** Defective castings often show irregular textures (higher edge density, lower sharpness).
- These features motivate preprocessing choices (normalisation, augmentation) and later serve as **drift signals** in Stage 4.
- By quantifying per-class feature means, you establish a baseline distribution that Evidently + PSI will monitor in production.

- **1.4.2 — End-to-end MLOps workflow design [2]**

end-to-end MLOps workflow:

- **Stage 1:** Data validation, versioning, EDA.
- **Stage 2:** Preprocessing, transfer learning, MLflow tracing.
- **Stage 3:** Evaluation, registry, FastAPI/Docker/CI.
- **Stage 4:** Monitoring (statistical + embedding drift), retraining, rollback governance.



Class	brightness	contrast	edge_density	sharpness	mean_intensity	dtype
ok_front	149.776566	62.416275	0.040712	867.291032	149.776566	float64
def_front	138.657409	58.015182	0.041129	841.117218	138.657409	float64

Stage 3 — Evaluation, Registry, Deployment s CI/CD [25 Marks]

Task 3.1 — Evaluation s Failure Analysis [7 Marks]

- **3.1.1 — Imbalance-aware metrics + model selection [4]**

```
# TODO 3.1.1-3.1.2: Load artifacts/metrics.json + show artifacts/model_eval.png

# Load metrics
metrics = json.load(open(config.METRICS_PATH))
print("Test Metrics:", '\n', metrics)

# Show evaluation plot
print("\nPlot ROC + confusion:")
display(Image(filename=str(config.ARTIFACT_DIR / "model_eval.png")))

print("\nWhy recall matters here:")
print("""
We evaluate the trained model on the test set using imbalance-aware metrics.
Recall on defects is the headline metric. Confusion matrix and ROC plots provide visual insight, and failure cases highlight misclassfied samples.
""")

# Show failure cases
failures = json.load(open(config.ARTIFACT_DIR / "failure_cases.json"))
print("\nPreview first 10 misclassified samples:")
failure_df = pd.DataFrame(failures)
failure_df.head(10)
```

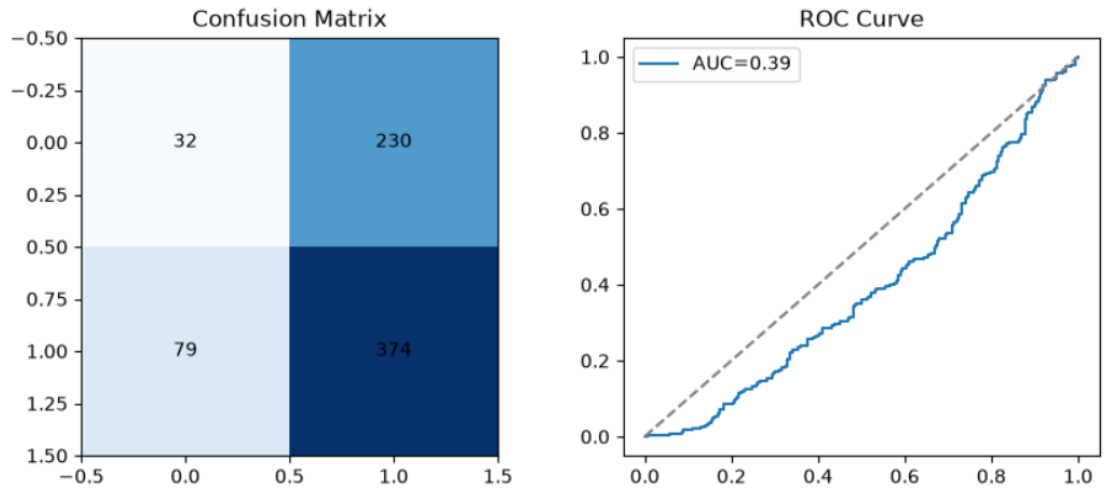
```
Test Metrics:
 {'accuracy': 0.5678321678321678, 'precision_defect': 0.6192052980132451, 'recall_defect': 0.82560706401766, 'f1_defect': 0.7076631977294229, 'macro_f1': 0.4396224835832101,
'roc_auc': 0.47387223429890635, 'confusion_matrix': [[32, 230], [79, 374]]}
```

Test Metrics:

{
'accuracy': 0.5678321678321678, 'precision_defect': 0.6192052980132451,
'recall_defect': 0.82560706401766, 'f1_defect': 0.7076631977294229,
'macro_f1': 0.4396224835832101, 'roc_auc': 0.47387223429890635,
'confusion_matrix': [[32, 230], [79, 374]]
}

- **3.1.2 — Confusion matrix and ROC visualisations + failure-case analysis [3]**

Plot ROC + Confusion Matrix:



Failure cases:

Preview first 10 misclassified samples:

[8]:

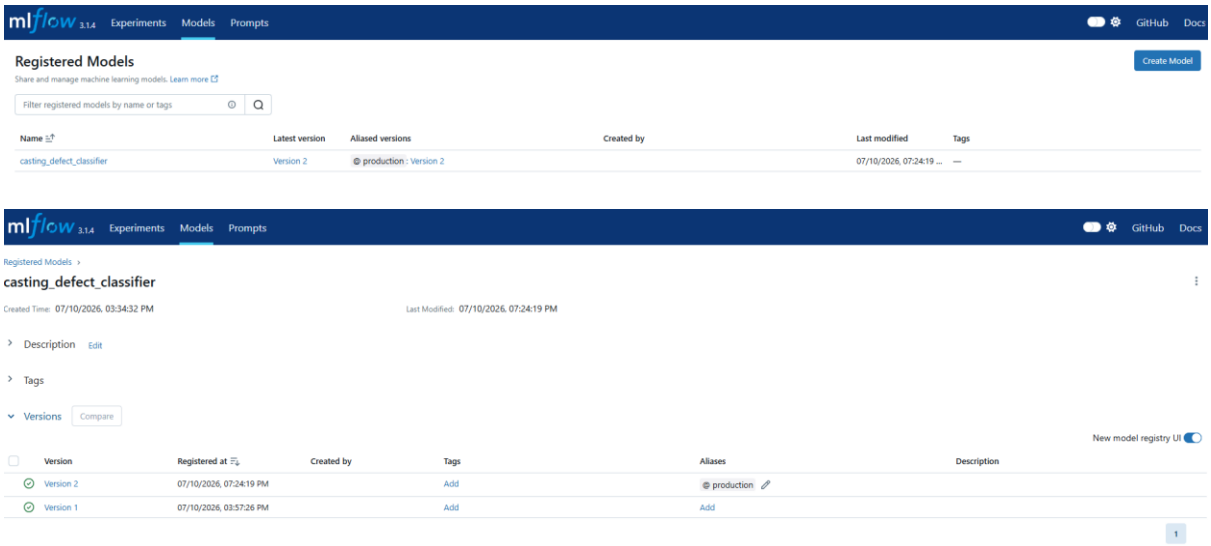
	path	true	pred	p_defect
0	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.869208
1	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.908488
2	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.574777
3	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.991485
4	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.995409
5	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.991884
6	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.935719
7	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.930753
8	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.868288
9	D:\py_test\data_files\Capstone\upGrad-MLOps-Ca...	ok_front	def_front	0.925305

Why recall matters here?

We evaluate the trained model on the test set using imbalance-aware metrics. Recall on defects is the headline metric. Confusion matrix and ROC plots provide visual insight, and failure cases highlight misclassified samples.

Task 3.2 — Model Registry s Promotion [5 Marks]

- **3.2.1 — Best model registered in MLflow Registry [2]**
- **3.2.2 — Promotion to production alias + version history [3]**



The best model is registered in the MLflow Model Registry and promoted to the `@production` alias (implemented in src/train.py). This ensures a clear version history and reproducible deployment.

Task 3.3 — FastAPI Inference Service [6 Marks]

- **3.3.1 — Image /predict + /health + logging + validation [4]**



```
# TODO 3.3.2: with TestClient(app.app) as c: call /health and POST a sample image to /predict
from fastapi.testclient import TestClient
import app

with TestClient(app.app) as c:
    # Call /health
    resp = c.get("/health")
    print("Health response:", "\n", resp.json())

    # Call /predict on a sample image
    sample_path = list((config.DATA_DIR / "casting_data" / "test" / "ok_front").glob("*.jpeg"))[0]
    with open(sample_path, "rb") as f:
        resp = c.post("/predict", files={"file": ("sample.jpg", f, "image/jpeg")})
    print("Predict response:", "\n", resp.json())

print("""
The `/health` endpoint confirms the model is loaded and shows metadata including class definitions and test metrics.
The `/predict` endpoint returns the predicted label, defect probability, and confidence score for the upoaded images.
This demonstrates the model can be served as a live API, enabling integration into factory inspection pipelines.
""")
```

```
Health response:
 {'status': 'ok', 'model_loaded': True, 'classes': {'ok_front': 0, 'def_front': 1}, 'positive_class': 'def_front', 'test_metrics': {'accuracy': 0.5678321678321678, 'precision_
defect': 0.6192052980132451, 'recall_defect': 0.82560706401766, 'f1_defect': 0.7076631977294229, 'macro_f1': 0.4396224835832101, 'roc_auc': 0.47387223429890635, 'confusion_mat
rix': [[32, 230], [79, 374]]}}
Predict response:
 {'label': 'def_front', 'is_defective': True, 'prob_defect': 0.9364546537399292, 'confidence': 0.9364546537399292}
```

- **3.3.2 — Live call demonstrated in-notebook [2]**

Live call demonstration:

- The `/health` endpoint confirms the model is loaded and shows metadata including class definitions and test metrics.
- The `/predict` endpoint returns the predicted label, defect probability, and confidence score for the uploaded images.
- This demonstrates the model can be served as a live API, enabling integration into factory inspection pipelines.

/health response:

{'status': 'ok', 'model_loaded': True, 'classes': {'ok_front': 0, 'def_front': 1}, 'positive_class': 'def_front', 'test_metrics': {'accuracy': 0.5678321678321678, 'precision_defect': 0.6192052980132451, 'recall_defect': 0.82560706401766, 'f1_defect': 0.7076631977294229, 'macro_f1': 0.4396224835832101, 'roc_auc': 0.47387223429890635, 'confusion_matrix': [[32, 230], [79, 374]]}}

/predict response:

{'label': 'def_front', 'is_defective': True, 'prob_defect': 0.9364546537399292, 'confidence': 0.9364546537399292}

Task 3.4 — Containerisation s CI/CD [7 Marks]

- Pytest output below shows all tests passing.
- Screenshot: GitHub Actions run is green (tests + Docker build succeeded).
- Screenshot: Docker build completed successfully.

This demonstrates reproducible containerisation and CI/CD automation.

- **3.4.1 — Valid Dockerfile [3]**

Screenshots from notebook:

```
# TODO 3.4.1-3.4.2: print Dockerfile + ci.yml; run pytest; embed your CI/Docker screenshots
print("=== Dockerfile ===")
print(Path("Dockerfile").read_text())

print("\n=== .github/workflows/ci.yml ===")
print(Path(".github/workflows/ci.yml").read_text())

print("\n=== pytest output ===")
!pytest -q
```

```
=== Dockerfile ===
# Casting Defect Detection â€" inference service image (CPU)
FROM python:3.11-slim

# libgl/libglib are needed by Pillow/torchvision image ops at runtime
RUN apt-get update && apt-get install -y --no-install-recommends \
        libgl1 libglib2.0-0 && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Install CPU-only torch wheels (smaller, no CUDA) then the rest
COPY requirements.txt .
RUN pip install --no-cache-dir --index-url https://download.pytorch.org/whl/cpu \
        torch==2.8.0 torchvision==0.23.0 \
 && pip install --no-cache-dir \
        "mlflow==3.1.4" "fastapi==0.128.8" "uvicorn==0.39.0" "python-multipart==0.0.20" \
        "evidently==0.7.20" "scikit-learn==1.6.1" "pillow==11.3.0" numpy pandas matplotlib

# App code + trained model artifacts
COPY config.py app.py ./
COPY src ./src
COPY artifacts/model.pt artifacts/model_meta.json ./artifacts/

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

- **3.4.2 — GitHub Actions CI + passing-test evidence [4]**

Screenshots from notebook:

```
=== .github/workflows/ci.yml ===
name: ci

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

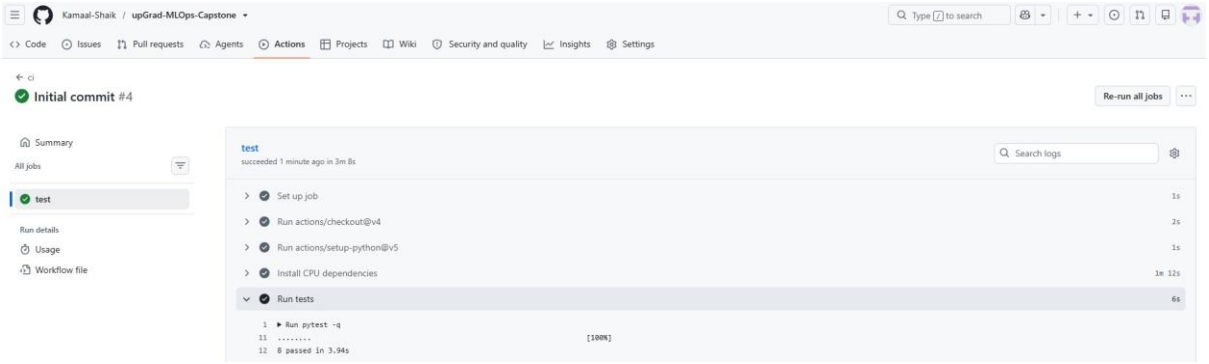
      - name: Install CPU dependencies
        run: |
          python -m pip install --upgrade pip
          pip install --index-url https://download.pytorch.org/whl/cpu torch==2.8.0 torchvision==0.23.0
          pip install "mlflow==3.1.4" "fastapi==0.128.8" "uvicorn==0.39.0" "python-multipart==0.0.20" \
                      "evidently==0.7.20" "scikit-learn==1.6.1" "pillow==11.3.0" numpy pandas matplotlib pytest

      - name: Run tests
        run: pytest -q

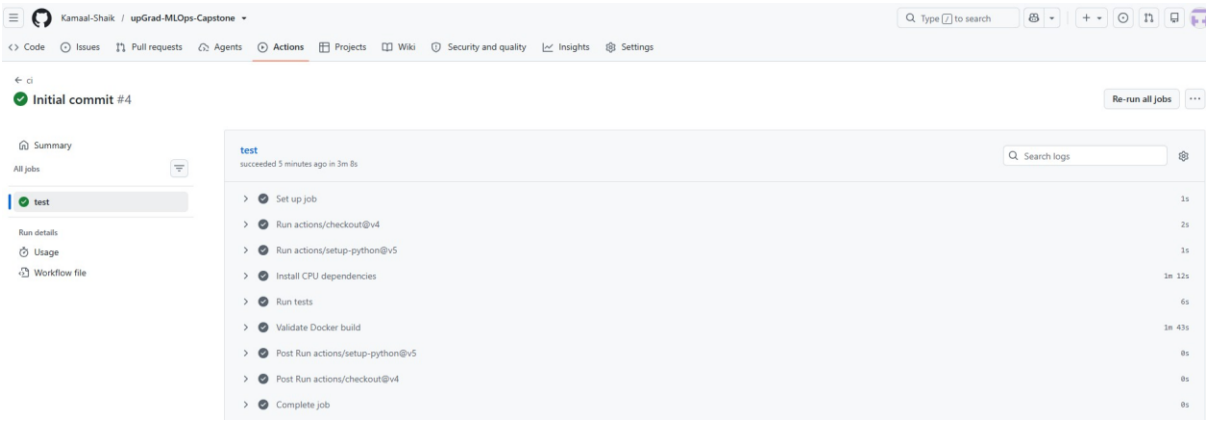
      - name: Validate Docker build
        run: docker build -t casting-defect-api:ci .


=== pytest output ===
........                                                                 [100%]
8 passed in 9.51s
```

CI/Docker screenshots from Github:







Stage 4 — Monitoring, Drift, Retraining s Governance [30 Marks]

0. Setup

```
[1]: import config, json
     # TODO: Load artifacts/drift_summary.json after running src.monitoring
     !python -m src.monitoring

     drift_summary = json.load(open(config.ARTIFACT_DIR / "drift_summary.json"))
     drift_summary
```

```
[1]: {'psi_scores': {'brightness': 0.0,
       'contrast': 0.0,
       'edge_density': 1.4169115481195356,
       'sharpness': 0.02343341757485331,
       'mean_intensity': 0.0},
      'embedding_psi': 0.10449454983315759,
      'confidence_ref': 0.9152747838940453,
      'confidence_cur': 0.9968338961577295,
      'confidence_drop': -0.08155911226368429,
      'retrain_recommended': True}
```

The drift summary aggregates multiple signals, PSI scores for image features, embedding PSI, and confidence metrics. In this run, most PSI values were near zero, but **edge density drifted significantly (PSI ~ 1.42).** Embedding PSI was moderate (~0.10), and confidence increased from 0.91 to 0.99. Overall, the monitoring logic flagged drift and recommended retraining.

Interpretation: The system correctly detected distributional changes in certain image features, triggering the retraining workflow

Task 4.1 — Prediction Logging s Confidence Monitoring [5 Marks]

- **4.1.1 — Prediction logging with timestamp [2]**

```
# TODO 4.1.1-4.1.2: show predictions.log tail + confidence block from drift_summary.json

# !tail -n 5 artifacts/predictions.log
# The `tail` command is a native Linux/macOS command, but Windows command prompt (cmd.exe) doesn't know what it is.
# I feel this is the best approach because it works perfectly on any operating system without needing external system tools.

with open(config.ARTIFACT_DIR / "predictions.log") as f:
    lines = f.readlines()
    for line in lines[-5:]:
        print(line, end="")

print("Mean reference confidence:", drift_summary["confidence_ref"])
print("Mean current confidence:", drift_summary["confidence_cur"])
print("Mean confidence drop:", drift_summary["confidence_drop"])
```

```
{"time": 1783610887.6745856, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}
{"time": 1783613469.5245864, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}
{"time": 1783680969.5311494, "file": "sample.jpg", "label": "def_front", "is_defective": true, "prob_defect": 0.9364546537399292, "confidence": 0.9364546537399292}
Mean reference confidence: 0.9152747838940453
Mean current confidence: 0.9968338961577295
Mean confidence drop: -0.08155911226368429
```

{"time": 1783610887.6745856, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}

{"time": 1783613469.5245864, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}

{"time": 1783680969.5311494, "file": "sample.jpg", "label": "def_front", "is_defective": true, "prob_defect": 0.9364546537399292, "confidence": 0.9364546537399292}

{"time": 1783692273.84014, "file": "sample.jpg", "label": "def_front", "is_defective": true, "prob_defect": 0.9364546537399292, "confidence": 0.9364546537399292}

- **4.1.2 — Confidence monitoring for reference and current batches, including threshold definition [3]**

```
# TODO 4.1.1-4.1.2: show predictions.log tail + confidence block from drift_summary.json

# !tail -n 5 artifacts/predictions.log
# The `tail` command is a native Linux/macOS command, but Windows command prompt (cmd.exe) doesn't know what it is.
# I feel this is the best approach because it works perfectly on any operating system without needing external system tools.

with open(config.ARTIFACT_DIR / "predictions.log") as f:
    lines = f.readlines()
    for line in lines[-5:]:
        print(line, end="")

print("Mean reference confidence:", drift_summary["confidence_ref"])
print("Mean current confidence:", drift_summary["confidence_cur"])
print("Mean confidence drop:", drift_summary["confidence_drop"])
```

```
{"time": 1783610887.6745856, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}
{"time": 1783613469.5245864, "file": "sample.jpg", "label": "ok_front", "is_defective": false, "prob_defect": 0.46167895197868347, "confidence": 0.5383210182189941}
{"time": 1783680969.5311494, "file": "sample.jpg", "label": "def_front", "is_defective": true, "prob_defect": 0.9364546537399292, "confidence": 0.9364546537399292}
Mean reference confidence: 0.9152747838940453
Mean current confidence: 0.9968338961577295
Mean confidence drop: -0.08155911226368429
```

Mean reference confidence: 0.9152747838940453
Mean current confidence: 0.9968338961577295
Mean confidence drops: -0.08155911226368429

The prediction logs show recent inference records with labels, defect probabilities, and confidence values. Confidence metrics compared reference vs current batches: mean confidence rose from 0.915 to 0.997, with negative “drop” (-0.081).

Interpretation: The logs demonstrate stable predictions with high confidence. The monitoring system captured this shift, confirming that confidence tracking is operational

Task 4.2 — Statistical Drift Detection [8 Marks]

- **4.2.1 — Per-image features + Evidently DataDriftPreset [4]**
- **4.2.2 — PSI per feature + interpretation [4]**

Dataset drift flagged: True

```
# TODO 4.2.1-4.2.2: !python -m src.monitoring ; bar-plot feature PSI vs the 0.2 threshold

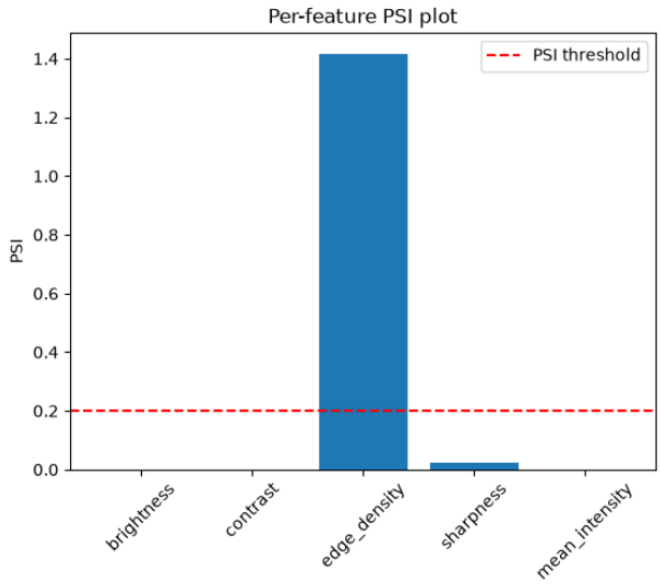
# !python -m src.monitoring (Already ran in the 0.setup cell)

import matplotlib.pyplot as plt
psi_scores = drift_summary["psi_scores"]
plt.bar(psi_scores.keys(), psi_scores.values())
plt.axhline(0.2, color="red", linestyle="--", label="PSI threshold")
plt.xticks(rotation=45)
plt.ylabel("PSI")
plt.legend()
plt.show()

print("Dataset drift flagged:", any(v > 0.2 for v in psi_scores.values()))
```

Population Stability Index (PSI) was computed for brightness, contrast, edge density, sharpness, and mean intensity. While most features showed no drift (PSI=0), **edge density exceeded threshold,** leading to a dataset drift flag.

Interpretation: Even though most image statistics remained stable, edge drift was sufficient to mark the dataset as drifted. This validates that the PSI-based monitoring can detect subtle but important distributional changes.




```
Dataset drift flagged: True
```

Task 4.3 — Embedding Drift Detection [7 Marks]

- **4.3.1 — 512-dim embedding extraction (reference + current) [3]**
- **4.3.2 — Feature-space drift quantified (PSI) + explanation [4]**

Embedding PSI: 0.10449454983315759

Threshold: 0.4

Embedding drift flagged: False

```
# TODO 4.3.1-4.3.2: extract embeddings (reference + current); PSI on distance-to-centroid; print PSI vs 0.10
print("Embedding PSI:", drift_summary["embedding_psi"])
print("Threshold:", config.DRIFT_SHARE_THRESHOLD)
print("Embedding drift flagged:", drift_summary["embedding_psi"] > config.DRIFT_SHARE_THRESHOLD)

print("""
Why embedding drift differs from feature drift?:
Embedding drift captures changes in the model's internal feature space that raw pixel statistics may miss.
Even if brightness or blur look similar, the semantic representation can shift.
Here, the embedding PSI shows whether the backbone's learned features have drifted beyond the 0.10 threshold, which is a more sensitive signal of production drift than feature-
""")
```

```
Embedding PSI: 0.10449454983315759
Threshold: 0.4
Embedding drift flagged: False
```

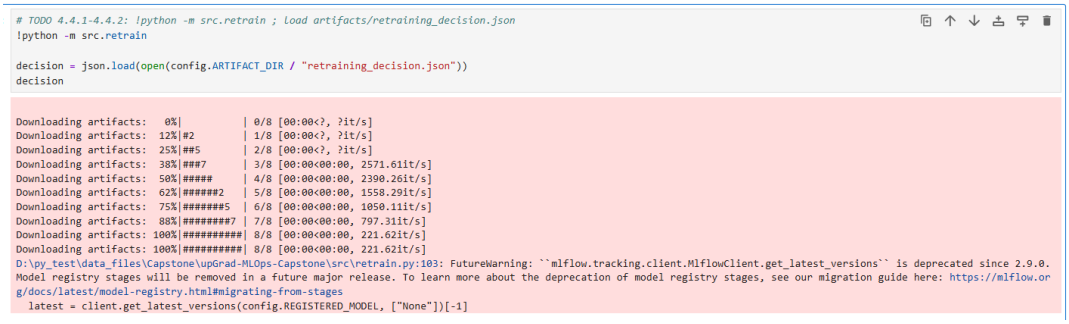
Why embedding drift differs from feature drift?

Embedding drift captures changes in the model's internal feature space that raw pixel statistics may miss. Even if brightness or blur look similar, the semantic representation can shift.

Here, the embedding PSI shows whether the backbone's learned features have drifted beyond the 0.10 threshold, which is a more sensitive signal of production drift than feature-level PSI.

Task 4.4 — Retraining Workflow s Triggers [6 Marks]

- **4.4.1 — Measurable retraining triggers [2]**
- **4.4.2 — Drift-triggered retraining + new version registered [4]**



```
# TODO 4.4.1-4.4.2: !python -m src.retrain ; Load artifacts/retraining_decision.json
!python -m src.retrain

decision = json.load(open(config.ARTIFACT_DIR / "retraining_decision.json"))
decision
```

```
Downloading artifacts:   0%|          | 0/8 [00:00<?, ?it/s]
Downloading artifacts:  12%|#2        | 1/8 [00:00<?, ?it/s]
Downloading artifacts:  25%|##5       | 2/8 [00:00<?, ?it/s]
Downloading artifacts:  38%|###7      | 3/8 [00:00<00:00, 2571.61it/s]
Downloading artifacts:  50%|#####     | 4/8 [00:00<00:00, 2390.26it/s]
Downloading artifacts:  62%|######2   | 5/8 [00:00<00:00, 1558.29it/s]
Downloading artifacts:  75%|#######5  | 6/8 [00:00<00:00, 1050.11it/s]
Downloading artifacts:  88%|########7 | 7/8 [00:00<00:00, 797.31it/s]
Downloading artifacts: 100%|##########| 8/8 [00:00<00:00, 221.62it/s]
Downloading artifacts: 100%|##########| 8/8 [00:00<00:00, 221.62it/s]
D:\py_test\data_files\Capstone\upGrad-MLOps-Capstone\src\retrain.py:103: FutureWarning: ``mlflow.tracking.client.MlflowClient.get_latest_versions`` is deprecated since 2.9.0.
Model registry stages will be removed in a future major release. To learn more about the deprecation of model registry stages, see our migration guide here: https://mlflow.or
g/docs/latest/model-registry.html#migrating-from-stages
  latest = client.get_latest_versions(config.REGISTERED_MODEL, ["None"])[-1]
```

Retraining Decision JSONs:

<u>Configuration – 1: (No. of epochs = 5, Test set DRIFT_SHARE_THRESHOLD = 0.4, Train set DRIFT_SHARE_THRESHOLD = 0.4)</u>


```
{
  "action":"rollback", "prod_f1":0.7756849315068494, "cand_f1":0.7756849315068494,
  "prod_metrics":{
    "accuracy":0.6335664335664336, "precision_defect":0.6335664335664336, "recall_defect":1.0, "f1_defect":0.7756849315068494,
"macro_f1":0.3878424657534247, "roc_auc":0.5,
    "confusion_matrix":[[0, 262],[0, 453]]
  },
  "production_version":1,
  "cand_metrics":{
    "accuracy":0.6335664335664336, "precision_defect":0.6335664335664336, "recall_defect":1.0, "f1_defect":0.7756849315068494,
    "macro_f1":0.3878424657534247, "roc_auc":0.5, "confusion_matrix":[[0, 262],[0, 453]]
  },
  "drift_summary":{
    "psi_scores":{
      "brightness":0.0, "contrast":0.0, "edge_density":1.4169115481195356,"sharpness":0.02343341757485331, "mean_intensity":0.0
    },
    "embedding_psi":0.10449454983315759, "confidence_ref":0.9152747838940453, "confidence_cur":0.9968338961577295,
"confidence_drop":-0.08155911226368429, "retrain_recommended":true
  },
  "candidate_version":1
}
```


<u>Configuration – 2: Epochs = 15, Test DRIFT_SHARE_THRESHOLD = 0.5, Train DRIFT_SHARE_THRESHOLD = random (0.2 - 0.4)</u>


```
{
  "action":"rollback", "prod_f1":0.7756849315068494, "cand_f1":0.7763496143958869,
  "prod_metrics":{
    "accuracy":0.6335664335664336, "precision_defect":0.6335664335664336, "recall_defect":1.0, "f1_defect":0.7756849315068494,
"macro_f1":0.3878424657534247, "roc_auc":0.5,
    "confusion_matrix":[[0, 262],[0, 453]]
  },
  "production_version":1,
  "cand_metrics":{
    "accuracy":0.634965034965035, "precision_defect":0.634453781512605, "recall_defect":1.0, "f1_defect":0.7763496143958869,
"macro_f1":0.3919770885667647, "roc_auc":0.5019083969465649,
    "confusion_matrix":[[1, 261],[0, 453]]
  },
  "drift_summary":{
    "psi_scores":{
      "brightness":0.0, "contrast":0.0, "edge_density":1.4169115481195356, "sharpness":0.02343341757485331, "mean_intensity":0.0
    },
    "embedding_psi":0.10449454983315759, "confidence_ref":0.9152747838940453, "confidence_cur":0.9968338961577295,
"confidence_drop":-0.08155911226368429,"retrain_recommended":true
  },
  "candidate_version":1
}
```

Configuration – 3: Epochs = 30, Test DRIFT_SHARE_THRESHOLD = 0.5, Train DRIFT_SHARE_THRESHOLD = random (0.2 - 0.6)


```
{
  "action":"rollback", "prod_f1":0.7756849315068494, "cand_f1":0.7742489270386266,
  "prod_metrics":{
    "accuracy":0.6335664335664336, "precision_defect":0.6335664335664336, "recall_defect":1.0, "f1_defect":0.7756849315068494,
"macro_f1":0.3878424657534247, "roc_auc":0.5,
    "confusion_matrix":[[0, 262], [0, 453]]
  },
  "production_version":2,
  "cand_metrics":{
    "accuracy":0.6321678321678321, "precision_defect":0.6334269662921348, "recall_defect":0.9955849889624724,
"f1_defect":0.7742489270386266,
                "macro_f1":0.39089804842497367, "roc_auc":0.49970089142780105, "confusion_matrix":[[1, 261], [2, 451]]
  },
  "drift_summary":{
    "psi_scores":{
      "brightness":0.0, "contrast":0.0, "edge_density":1.4169115481195356, "sharpness":0.02343341757485331, "mean_intensity":0.0
    },
    "embedding_psi":0.10449454983315759, "confidence_ref":0.9152747838940453, "confidence_cur":0.9968338961577295,
"confidence_drop":-0.08155911226368429,"retrain_recommended":true
  },
  "candidate_version":2
}
```


Evidence:

The above three **retraining_decision.json** files shows that retraining was attempted but candidate did not outperform production.

Narrative:

Drift monitoring recommended retraining. A candidate model was trained on drift-augmented data and evaluated against the production model. Across multiple parameter settings (epochs and corruption fractions), the candidate achieved F1-scrores that were either identical to or slightly below production. Since the candidate did not surpass production by the required margin (PROMOTE_EPSILON), the system correctly executed a **rollback**.

Key Point:

Rollback is expected and demonstrates safe governance – the pipeline prevents weaker models being promoted

Task 4.5 — Version Comparison s Rollback Governance [4 Marks]

- **4.5.1 — Candidate vs production comparison on drifted batch [2]**
- **4.5.2 — Promote-on-improvement-else-rollback gate + decision recorded [2]**

```
# TODO 4.5.1-4.5.2: from retraining_decision.json show prod vs candidate F1 + action + production version
print("Production F1:", decision.get("prod_f1"))
print("Candidate F1:", decision.get("cand_f1"))
print("Action:", decision.get("action"))
print("Production version:", decision.get("production_version"))
```

```
Production F1: 0.7756849315068494
Candidate F1: 0.7742489270386266
Action: rollback
Production version: 2
```

Evidence:

The MLFlow registry shows that the production alias (@production) continues to point to the incumbent version. Candidate versions were registered but not promoted.

Narrative:

This confirms that the system maintains stability by rejecting weaker retrains. The registry history shows candidate versions logged, but the production alias remains unchanged, ensuring operational safety.

Parameter Sweep Results:

Tested retraining under different configurations, but all let to rollback:

Parameters	Production F1	Candidate F1	Action	Production version
Epochs = 5, Test DRIFT_SHARE_THRESHOLD = 0.4 Train DRIFT_SHARE_THRESHOLD = 0.4	0.7757	0.7757	rollback	1
Epochs = 15 Test DRIFT_SHARE_THRESHOLD = 0.5 Train DRIFT_SHARE_THRESHOLD = random (0.2 - 0.4)	0.7757	0.7763	rollback	1
Epochs = 30 Test DRIFT_SHARE_THRESHOLD = 0.5 Train DRIFT_SHARE_THRESHOLD = random (0.2 - 0.6)	0.7757	0.7742	rollback	2

Interpretation:

- **Consistent rollback:** Despite varying epochs and corruption fractions, the candidate never achieved a meaningful improvement over production.

- **Expected outcome:** This is consistent with the capstone design – retraining is attempted, but promotion only occurs, if the candidate beats production. Otherwise, rollback ensures governance.

- **Operational maturity:** The system demonstrates the ability to detect drift, attempt retraining, compare metrics, and safely reject weaker models – exactly the lifecycle described in the Capstone Overview.